Lab 1
GPT-2: Load the GPT-2 Hugging Face model, analyze its structure and generate text

Laurea Magistrale in Artificial Intelligence, University of Verona
Course: Natural Language Processing (topic: Large Language Models)
Alberto Castellini

1. The first publication of GPT-2

OpenAI Article, February 14, 2019: Better language models and their implications.
https://openai.com/index/better-language-models/

Main points:

- We've **trained a large-scale unsupervised language model** which **generates coherent paragraphs of text**, achieves **state-of-the-art performance on many language modeling benchmarks**, and performs rudimentary **reading comprehension**, **machine translation**, **question answering**, and **summarization**—all **without task-specific training**.

- Our model, called GPT-2 (a successor to GPT), was trained simply to **predict the next word** in **40GB of Internet text**. Due to our concerns about malicious applications of the technology, we are not releasing the trained model. As an experiment in responsible disclosure, **we are instead releasing a much smaller model [GPT-2 Github]** for researchers to experiment with, as well as a technical paper [GPT-2 Paper].

- GPT-2 is a **large transformer-based language model** with **1.5 billion parameters**, trained on a **dataset of 8 million web pages**. GPT-2 is trained with a simple **objective: predict the next word**, **given all of the previous words within some text**. The diversity of the dataset causes this simple goal to contain **naturally occurring demonstrations of many tasks across diverse domains**. GPT-2 is a direct scale-up of GPT, with more than 10X the parameters and trained on more than 10X the amount of data.

- GPT-2 displays a broad set of capabilities, including the ability to **generate conditional synthetic text samples** of unprecedented quality, where we prime the model with an input and have it generate a **lengthy continuation**. In addition, GPT-2 **outperforms** other language models trained on **specific domains** (like Wikipedia, news, or books) without needing to use these domain-specific training datasets. On language tasks like question answering, reading comprehension, summarization, and translation, **GPT-2 begins to learn these tasks from the raw text**, using **no task-specific training data**. While scores on these downstream tasks are far from state-of-the-art, they suggest that the tasks **can benefit from unsupervised techniques**, given sufficient (unlabeled) data and compute.

- GPT-2 **generates synthetic text samples** in response to the model being primed with an **arbitrary input**. The model is chameleon-like—**it adapts to the style and content of the conditioning text**. This allows the user to **generate realistic and coherent continuations** about a topic of their choosing, as seen by the following select samples.

- **Zero-shot:** GPT-2 achieves state-of-the-art scores on a **variety of domain-specific language modeling tasks**. Our model is not trained on any of the data specific to any of

these tasks and is only evaluated on them as a final test; this is known as the **"zero-shot"** setting. GPT-2 **outperforms models trained on domain-specific datasets** (e.g. Wikipedia, news, books) when evaluated on those same datasets. The following table shows all our state-of-the-art zero-shot results.

- On **other language tasks** like **question answering**, **reading comprehension**, **summarization**, and **translation**, we are able to get **surprising results without any fine-tuning** of our models, simply by **prompting** the trained model in the right way (see below for examples of how we do this), though we do still fall short of state-of-the-art for specialized systems.

2. GPT-2: The official paper

GPT-2 Paper: Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever. **Language Models are Unsupervised Multitask Learners**, 2019, (https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf)

Main points:

- The GPT-2 model generates new text by predicting the probability of the next tokens

$$p(x) = \prod_{i=1}^{n} p(s_n | s_1, ..., s_{n-1})$$

- Performance of GPT-2 on different tasks depending on the size

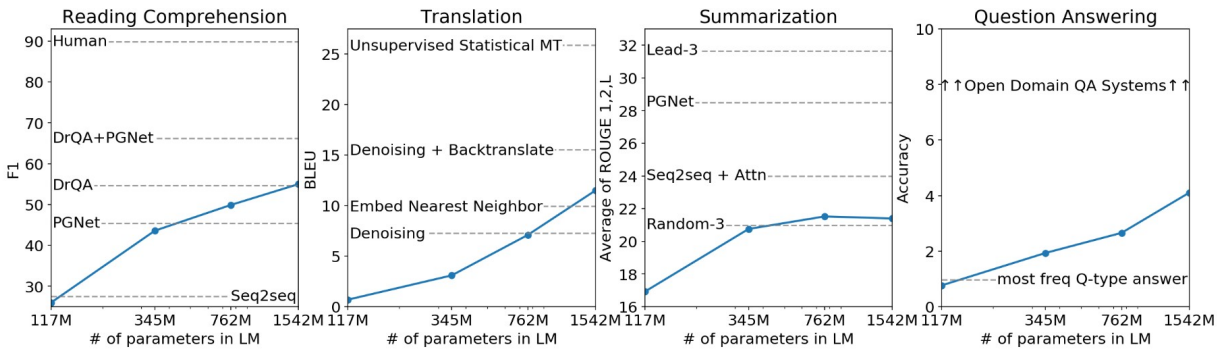


Figure 1. Zero-shot task performance of WebText LMs as a function of model size on many NLP tasks. Reading Comprehension results are on CoQA (Reddy et al., 2018), translation on WMT-14 Fr-En (Artetxe et al., 2017), summarization on CNN and Daily Mail (See et al., 2017), and Question Answering on Natural Questions (Kwiatkowski et al., 2019). Section 3 contains detailed descriptions of each result.

- Network Sizes of different GPT-2 versions

Parameters	Layers	d_{model}
117M	12	768
345M	24	1024
762M	36	1280
1542M	48	1600

Table 2. Architecture hyperparameters for the 4 model sizes.

Errata corrige: 117M → 124M

3. A quick look to the GPT-3 paper

GPT3 Paper: Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei. **Language Models are Few-Shot Learners,** 2020 (https://arxiv.org/abs/2005.14165)

The GPT-2 paper is not very technically precise so we'll reference also the GPT3 paper.

4. The goal of our lab on GPT-2

The goals of the three lab sessions about GPT-2 are:

- To **use** the pre-trained GPT-2 model available online
- To **re-implement** the GPT-2 124M model
- To understand how to **train** it from scratch (pre-training)

Nowadays it is possible to train a GPT-2 model in about 1 hour and spending about 20$ using cloud resources.

OpenAI released the weights for GPT-2. You can find models in Hugging Face and [GPT-2 Github]

5. The GPT-2 124M model released by OpenAI

- The model uses **Tensorflow** https://github.com/openai/gpt-2/blob/master/src/model.py which is currently used less than PyTorch
- We'll reproduce the model using **PyTorch**
- We'll use the **HuggingFace** transformers
  - https://github.com/huggingface/transformers
  - https://github.com/huggingface/transformers/tree/main/src/transformers
- Which provide several models
  - https://github.com/huggingface/transformers/tree/main/src/transformers/models
- One of them is GPT-2
  - https://github.com/huggingface/transformers/tree/main/src/transformers/models/gpt2
- The *modeling_gpt2.py* file https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py contains the implementation of the GPT-2 model. Try to analyze it.
- It is readable but it contains some optimizations that make the code complex.

6. Let's run Hugging Face GPT-2 model

Prepare the conda environment

- Download miniconda
  >> wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
- Install it (download file *build-nanogpt-conda-environment.yml* from Moodle)
  >> bash Miniconda3-latest-Linux-x86_64.sh
  >> source $HOME/miniconda3/etc/profile.d/conda.sh
  >> conda init
- Install and activate the conda environment *build-nanogpt* for the project (yml file attached)
  >> conda env create -f build-nanogpt-conda-environment.yml
- Check the list of conda environments
  >> conda env list
- Activate the conda environment
  >> conda activate build-nanogpt
- Install the missing packages
  >> pip install ipykernel
  >> pip install matplotlib
- Deactivate the conda environment
  >> conda deactivate

Install Visual Studio Code:

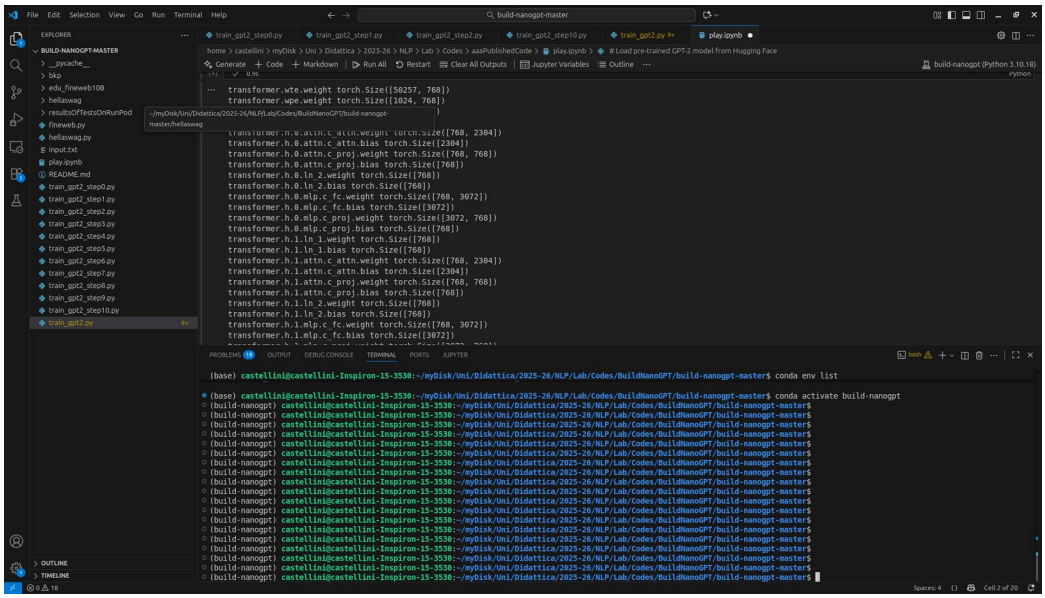
- Download and install the IDE (https://code.visualstudio.com/)
- Open the Extensions panel (Ctrl+Shift+X)
- Search for *Python* and *Jupyter* extensions
- Click Install on both

Open VS Code

Open the notebook: use the file explorer to search and open the file *play_Template.ipynb*

Run the notebook

- Click on the play icon of each cell to run it.
- The first time you do it, VS Code will ask you (on top of the screen) to select the conda environment. Select the *build-nanogpt* environment.



7. Use the notebook to analyze the pre-trained GPT-2 model

- Analyze the notebook
- The GPT-2 model has the following **structure**:

  GPT2Model
  └── transformer
  ├── **wte** (token embeddings) → From token dictionary to token embeddings
  ├── **wpe** (positional embeddings) → Positional embedding
  ├── **h[N]** (list of transformer blocks) → Transformer layers (**n_layer=12**)
  └── **ln_f** (final layer norm) → final layer normalization to get probabilities

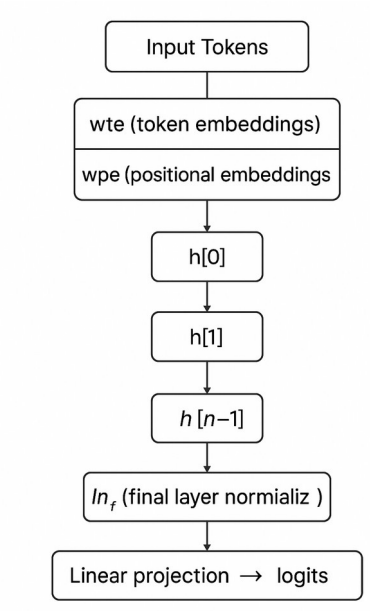
- Each GPT-2 block (**transformer.h[i]**) contains four main **components**:
  - **ln_1** → Layer normalization before attention
  - **attn** → Multi-head self-attention
  - **ln_2** → Layer normalization before feed-forward
  - **mlp** → Feed-forward (two linear layers with GELU activation)

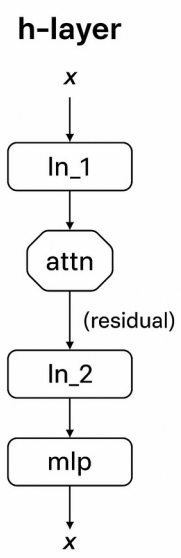
- **Question 1:** Visualize and understand the **model elements (wte, wpe, h[i] i=0,..,11, ln_f, lm_head)**: [Answer already in the notebook]

```
transformer.wte.weight torch.Size([50257, 768])
transformer.wpe.weight torch.Size([1024, 768])
transformer.h.0.ln_1.weight torch.Size([768])
transformer.h.0.ln_1.bias torch.Size([768])
transformer.h.0.attn.c_attn.weight torch.Size([768, 2304])
transformer.h.0.attn.c_attn.bias torch.Size([2304])
transformer.h.0.attn.c_proj.weight torch.Size([768, 768])
transformer.h.0.attn.c_proj.bias torch.Size([768])
transformer.h.0.ln_2.weight torch.Size([768])
transformer.h.0.ln_2.bias torch.Size([768])
transformer.h.0.mlp.c_fc.weight torch.Size([768, 3072])
transformer.h.0.mlp.c_fc.bias torch.Size([3072])
transformer.h.0.mlp.c_proj.weight torch.Size([3072, 768])
transformer.h.0.mlp.c_proj.bias torch.Size([768])
transformer.h.1.ln_1.weight torch.Size([768])
transformer.h.1.ln_1.bias torch.Size([768])
transformer.h.1.attn.c_attn.weight torch.Size([768, 2304])
transformer.h.1.attn.c_attn.bias torch.Size([2304])
transformer.h.1.attn.c_proj.weight torch.Size([768, 768])
transformer.h.1.attn.c_proj.bias torch.Size([768])
transformer.h.1.ln_2.weight torch.Size([768])
transformer.h.1.ln_2.bias torch.Size([768])
transformer.h.1.mlp.c_fc.weight torch.Size([768, 3072])
transformer.h.1.mlp.c_fc.bias torch.Size([3072])
transformer.h.1.mlp.c_proj.weight torch.Size([3072, 768])
...
transformer.h.11.mlp.c_proj.bias torch.Size([768])
transformer.ln_f.weight torch.Size([768])
transformer.ln_f.bias torch.Size([768])
lm_head.weight torch.Size([50257, 768])
```

The complete structure is the following

```
Input tokens
  ↓
Token embedding (wte)
  ↓
Positional embedding (wpe)
  ↓
N × Transformer blocks (h[0], h[1], …, h[N-1])
  ↓
Final LayerNorm (ln_f)
  ↓
Linear output head (lm_head)
  ↓
Logits (vocabulary probabilities)
```

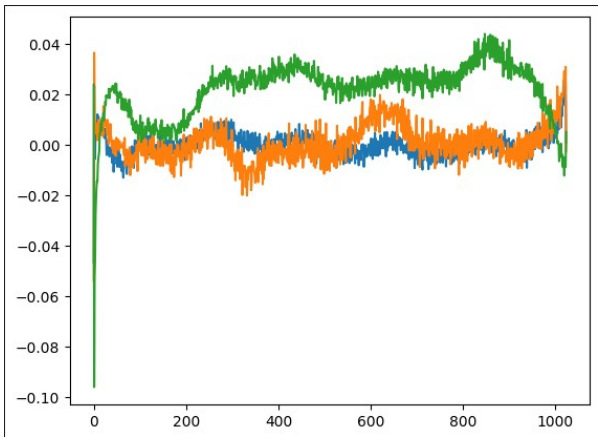




- transformer.**wte**.weight torch.Size([50257, 768])
  - → wte=**weights for token embedding**
  - → 50257 is the dimension (# tokens) in the GPT-2 vocabulary (vocab_size)
  - → 768 is the model dimension, namely the token embedding dimension

- transformer.**wpe**.weight torch.Size([1024, 768])
  - → wpe=**weights for position embedding**
  - → 1024 is the maximum sequence length (context length) of GPT-2
  - → 768 is the length of the vector learned for each position of the context to identify it

- **Question 2:** Visualize the **positional embeddings as a gray-scale image**
  - → Each row is the representation of a specific position from 0 to 1023 in the GPT-2 input sequence, each column is a variable in the embedding space
- **Question 3:** Visualize some rows of the positional embeddings (rows of transformer.wpe.weight)
- **Question 4:** Visualize some columns of the positional embeddings (columns of transformer.wpe.weight)
  - → In different positions each of the 768 channels respond more or less to that position



- The wpe table is initialized randomly and as it is trained the column lines become more smooth. Now it has some sinusoidal-like shape

- Analyze and understand the **model elements (h.i, i=0..11)**:
  - Each transformer layer h.i (i=1..11) has:
    - → transformer.h.i.**ln_1**.weight torch.Size([768])
    - → transformer.h.i.**ln_1**.bias torch.Size([768])
      - **layer normalization (weights and biases)** applied to the input of the attention block. LayerNorm normalizes each token's hidden representation across its 768 features, stabilizing training. For input vector $x \in \mathbb{R}^{768}$:

$$LN(x) = (x - mean(x)) / sqrt(var(x) + \varepsilon) * weight + bias$$

Question 5: Compare the position (in the overall architecture) of the layer normalization in the Attention is all you need paper [Transformers Paper] and the GPT-2 model. Are them the same?

→ transformer.h.i.**attn.c_attn**.weight torch.Size([768, 2304])
→ transformer.h.i.**attn.c_attn**.bias torch.Size([2304])
  - Single linear layer that computes all three of the self-attention projections:
    - Q (queries) [768, 768]
    - K (keys) [768, 768]
    - V (values) [768, 768]

    Instead of having three separate weight matrices, GPT-2 combines them into one big matrix [768, 2304].

    $$[Q, K, V] = X * W_c_attn + b_c_attn$$

    where $W_c_attn \in \mathbb{R}^{(768\times2304)}$ and $b_c_attn \in \mathbb{R}^{(2304)}$.

→ transformer.h.i.**attn.c_proj**.weight torch.Size([768, 768])
→ transformer.h.i.**attn.c_proj**.bias torch.Size([768])
  - After multi-head attention computes its outputs (concatenating all attention heads), GPT-2 projects the result back into the embedding dimension using another linear layer, c_proj.
  - Input = concatenated attention heads → size 768. Output = same size 768. Projection matrix is [768, 768].

    $$Y = AttentionOutput * W_c_proj + b_c_proj$$

→ transformer.h.0.**ln_2**.weight torch.Size([768])
→ transformer.h.0.**ln_2**.bias torch.Size([768])
  - Another layer normalization, applied before the feed-forward (MLP) sublayer. Same as ln_1.

→ transformer.h.0.**mlp.c_fc**.weight torch.Size([768, 3072])
→ transformer.h.0.**mlp.c_fc**.bias torch.Size([3072])
  - First linear layer. This is the "position-wise" feed-forward network - it operates on each token separately (no attention here). Expands the embedding dimension from 768 to 3072, increasing capacity. The activation (typically GELU) follows this layer.

    $$H = GELU(X * W_c_fc + b_c_fc)$$

→ transformer.h.0.**mlp.c_proj**.weight torch.Size([3072, 768])
→ transformer.h.0.**mlp.c_proj**.bias torch.Size([768])
  - Second linear layer. Projects the expanded MLP output back to the embedding dimension (3072 → 768).

    $$Y = H * W_c_proj + b_c_proj$$

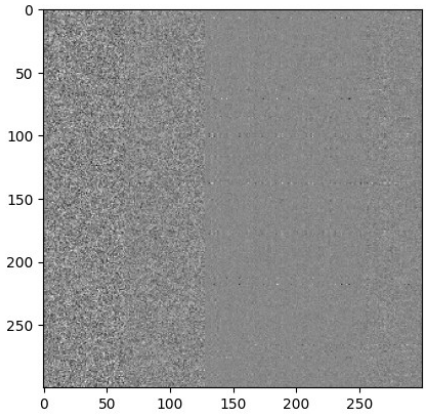
► **Notice:** the **multi-head structure** is not explicitly visible in the model structure but it is implicit in how the weights of c_attn and c_proj are shaped and used during the forward pass. See the forward() method of the attention module (GPT2Attention).

- Visualize and understand the **model elements (transformer.ln_f and lm_head.weight)**:
  ► transformer.**ln_f**.weight torch.Size([768])
  ► transformer.**ln_f**.bias torch.Size([768])
    → Final layer normalization - the "ln_f" in GPT-2 stands for "layer normalization final." See previous layer normalization. It helps i) stabilize training and gradient flow, ii)

Ensure consistent statistics across tokens before the output projection, iii) Improve generalization.

► lm_head.weight torch.Size([50257, 768])
→ It is the **output layer** that turns GPT-2's hidden representations into actual **token probabilities over the vocabulary**.

- **Question 6:** Visualize part of the attention weights (matrices W_Q, W_K and W_V) of the first attention layer



- Use the model and the pipeline('text-generation', model='gpt2') function to generate new text given the prompt "Hello, I'm a language model,"

```
[{'generated_text': "Hello, I'm a language model, I'm a language model. In my mind, I'm doing the same thing as you. All these differ
 {'generated_text': 'Hello, I\'m a language model, and I\'m trying to understand what you\'re saying. In English, I have two English
 {'generated_text': "Hello, I'm a language model, so I don't get much of a chance to interact with objects. But I did find that I can
 {'generated_text': 'Hello, I\'m a language model, a language model, a language model."'},
 {'generated_text': "Hello, I'm a language model, not a toolkit.\n\nIn a nutshell, I need to give language models a set of properties
```

- Check the final code to understand how the tokens can be sampled manually

- Play with the notebook. Enjoy!

References

- [GPT2 Github] https://github.com/openai/gpt-2
- [GPT2 Paper] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever. Language Models are Unsupervised Multitask Learners, 2019, (https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf)
- [GPT3 Paper] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei. Language Models are Few-Shot Learners, 2020 (https://arxiv.org/abs/2005.14165)
- [Transformers Paper] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin. Attention Is All You Need , 2017 (https://arxiv.org/abs/1706.03762)
- File play.ipynb from https://github.com/karpathy